Tennis match prediction with machine learning

Simone Bonasorta
Liceo Scientifico Nicola Pizi, Palmi

Abstract

I made this project for amateur reasons and for fun, to grow my knowledge of machine learning, that until now was limited to Anomaly Detection and not to win prediction. The idea is that the model works on thousands of matches collected from the early 2000s (2005 to 2026) up to today. The goal is to estimate the win probability of a player using only the statistics of his past matches, the ELO score, and a rating that can be compared on a global scale. The model predicts before the match starts, based on who the players are and the context they play in, nothing more and nothing less; the strong part are the features I extract and use. In this paper I show how I built the model, how it compares to the best amateur attempt I know about, and how the accuracy relates to the probabilities used by the BookMaker, the AI that runs on betting sites when you bet pre-match or in-match and tells how much one bet should pay with respect to another.

INDICES

1. INTRODUCTION

I made this project for amateur reasons and for fun, to grow my knowledge of machine learning, that until now was limited to Anomaly Detection and not to win prediction. Tennis is a good problem for this: the data is clean, the result is binary (one player wins, the other loses) and public datasets exist with more than twenty years of matches.

My starting question was simple: how high can I push the accuracy of a tennis predictor, using only free public data? Public attempts I found online, in particular the one I describe in the next section, stop around **60–63% accuracy**. That number became my reference: I wanted to see if the limit comes from the data or from the techniques used, and how much further you can go when the pipeline is done well.

I want to be clear on one thing from the start. The output of my model is a *probability*, not a bet. I never used it to place real bets and I never tried to tune it for profit. The right point of comparison for these probabilities is the BookMaker, that I describe in the next section.

2. RELATED WORK

2.1 Public amateur baseline

The clearest amateur attempt I know about is the YouTube tutorial by the channel *Green Code* [7], that builds a Random Forest on a small set of features (ELO ratings, ranking, surface) and reaches around **60–63% accuracy** on its test set. This is my reference: any honest improvement to the pipeline should show as a clear gain over this number, on data the model has not seen during training.

2.2 The BookMaker

The BookMaker is the AI that runs on betting sites: when you bet, both pre-match and in-match, it tells how much one bet should pay with respect to another. In practice it gives a probability for each player, and if a lot of money goes on one side the probability of that side goes up and the payout on the other side goes down.

Given two decimal odds o_1, o_2, the implied probabilities, after removing the margin of the bookmaker, are

$$\hat{p}_1 = \frac{1/o_1}{1/o_1 + 1/o_2}, \qquad \hat{p}_2 = 1 - \hat{p}_1. \tag{1}$$

The number $\hat{p}_1$ is the probability that player 1 wins according to the BookMaker. This is the natural number to compare my model against, because it is also a probability, set by a serious system, on the same matches.

In the rest of the paper I use the closing odds of *Pinnacle*, a sharp bookmaker that is known to set accurate prices on the ATP main tour.

3. METHODOLOGY

3.1 Strategy

I chose to work only on **pre-match** prediction. This means that for every match the model can only see what was known *before* the first ball was played: rankings, surface, recent results, statistics, head-to-head, and the closing odds (that are also fixed before the start of the match). I did not use in-play data, like live score or live odds.

The reason is mostly about method. Pre-match prediction is the cleanest case to study, because the inputs are well defined and the risk of leaking the result is much smaller. It is also the same setting used by the amateur baseline I want to compare with, so the comparison is fair.

Inside this setting, my strategy was to start from a simple ELO baseline and add features one by one, measuring each time the gain on a fixed validation set. This way I always knew how much each piece was bringing, and I avoided the typical mistake of stacking many techniques together without understanding what they actually do.

3.2 Architecture

The model is a gradient boosted tree ensemble, made with **XGBoost** [4]. I did not use a neural network: with around 40 000 matches and hand-built features, boosted trees usually do at least as well as a neural network, and the feature importance is much easier to read. This mattered to me, because I wanted to check at each step that the model was really using the features I expected, and not random patterns.

On top of the raw XGBoost output, I applied an **isotonic regression** calibrator, fitted on the validation set. Calibration does not change which player is predicted as the winner, but it makes the probability trustable: when the calibrated model says "80%", the player actually wins around 80% of the times. This is important when you want to compare the model with the BookMaker, that already gives you a probability.

I also tried, as a small extension, to stack a second model on top of the XGBoost output and the ELO baseline, using out-of-fold predictions and a meta logistic regression. The gain on the test set was below half a point, so I kept the calibrated XGBoost as the main model.

3.3 Data Collection

The main data source is the `tennis_atp` repository of Jeff Sackmann [1], free on GitHub. It contains all ATP matches from the early 2000s, with rankings, surface, tournament level, round, scores and serve statistics per match. After loading all the yearly CSV files I got around 63 000 matches between 2005 and 2026.

For the BookMaker reference I used historical closing odds from *tennis-data.co.uk* [2], that publishes weekly Excel files with the closing prices of many bookmakers, in particular Pinnacle and Bet365. I was able to attach closing odds to around 33 000 matches, mostly from 2013 on, and only for the ATP main tour.

The split is strictly in time. The first seasons are used as a burn-in to let the ELO ratings stabilise, the central seasons are the training set, the next season is the validation, and the most recent seasons (2025 and the part of 2026 already played) are the held-out test. A random split would let the model see future information of the same player, and the accuracy would be wrong without me noticing. This is exactly the kind of mistake the project tries to avoid.

3.4 Data Preprocessing

A few preprocessing steps were applied before training:

- **Row filtering:** I removed walkovers and matches retired before the first game, because they are not a real tennis result.
- **Player swap randomisation:** for every match I decided in a random but reproducible way (seeded by date and match number) if the winner is labelled as "player 1" or "player 2". Without this step the model would just learn the trivial rule "player 1 always wins" and the accuracy would mean nothing.
- **State-before-match rule:** for every match, the features are computed from the state of the world *before* the match starts, and the state is updated only after. This rule is the most important step of the whole pipeline: if you do it in the wrong order, the result leaks into the features and the accuracy becomes fake.
- **Splitting:** the cleaned dataset was split by date into training, validation and test, with the validation used both for early stopping and for the calibrator.

3.5 Features Used

The feature vector of each match is made of five groups.

ELO features. The first feature is the ELO rating itself, that gives a single number for the strength of a player. I keep a general ELO rating and a surface-specific one (Hard, Clay, Grass, Carpet) [3]. To turn the ELO ratings of the two players into a probability before the match, I use the standard ELO formula

$$E_A = \frac{1}{1 + 10^{(R_B - R_A)/400}}, \tag{2}$$

which says that if $R_A = R_B$ the probability is 0.5, and that every 400 points of difference multiply the odds by 10. After the match the ratings are updated by

$$R'_A = R_A + K\,(S_A - E_A), \tag{3}$$

where $S_A \in \{0, 1\}$ is the result of the match and K is a learning rate that goes down with the number of matches played, so that young players move faster than veterans. I use the ratings, the surface ratings and their differences as features.

Ranking features. I use the official ATP ranking and the ranking points of both players, and their differences. To make the behaviour more stable for very different rankings (like a top 5 against a top 300) I also use log-transformed differences, because raw differences in those cases would be huge and not really informative.

Recent form and head-to-head. For each player I compute the fraction of wins in the last 10 matches, separately overall and on the same surface, and the head-to-head record between the two players, both in general and on the same surface only.

Serve and physical features. From the serve statistics of the Sackmann dataset I build rolling averages of first-serve percentage, points won on first serve, points won on second serve, ace and double-fault rates, and break points saved. I also compute a fatigue score, defined as the total number of minutes played in the last 14 days.

BookMaker-implied probability. As the last group of features I add the implied probability of player 1 from the closing odds, computed with the formula of Section 2.2. It is assigned to player 1 with the same deterministic swap, not based on the actual result, so it does not leak the outcome.

4. Results

4.1 Training Parameters

The XGBoost model was trained with log-loss as the objective and with early stopping on the validation set. On top of the raw scores I applied an isotonic regression calibrator fitted on the same validation set. The final calibrated model has around 40–60 boosting rounds, depending on the training run, and a maximum tree depth of 6.

4.2 Evaluation metrics

To measure how well the model classifies the matches I use the standard binary classification metrics. The first one is the accuracy, that counts how often the model picks the right winner over all the matches:

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{Total}}. \tag{4}$$

Accuracy alone is not enough, because it does not tell if the model is overconfident on one of the two classes. So I also use precision and recall, that look at the wins of player 1 only:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \tag{5}$$

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \tag{6}$$

Table 1: Test-set metrics.

Metric	ELO baseline	My model
Accuracy	0.634	**0.676**
Precision	0.633	**0.677**
Recall	0.640	**0.678**
F1 score	0.636	**0.677**
Brier	0.221	**0.207**
Log-loss	0.630	**0.599**

and the F1 score, that puts them together as the harmonic mean:

$$\text{F1} = 2\,\frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}. \tag{7}$$

For the quality of the predicted probabilities I also report the Brier score [6], that is the mean squared error between the predicted probability and the actual result (1 if player 1 wins, 0 otherwise):

$$\text{Brier} = \frac{1}{N}\sum_{i=1}^{N}(p_i - y_i)^2, \tag{8}$$

and the log-loss, that punishes confident wrong predictions much harder than uncertain wrong predictions:

$$\mathscr{L} = -\frac{1}{N}\sum_{i=1}^{N}\big[y_i \log p_i + (1-y_i)\log(1-p_i)\big]. \tag{9}$$

For both Brier and log-loss, lower values are better.

4.3 Model performance

Table 1 shows the metrics of the calibrated XGBoost model on the test set (more than 4000 matches from the most recent seasons), against the pure ELO baseline I started from.

Compared with the $\sim 63\%$ accuracy of the amateur baseline [7], my model gains around **4 percentage points**, which in this setting is a clear jump. To put a number on the uncertainty, I bootstrapped the test set (resampling matches with replacement, 2000 times): the 95% confidence interval on accuracy is **[0.665, 0.693]**. The interval is tight because the test set is large (more than 4000 matches), unlike a single-season sport, so the improvement over the baseline is not just noise.

4.4 Ablation

To see how much each kind of information actually contributes, I removed one feature group at a time, retrained the model and re-measured the test accuracy (Table 2). The result is clear and, for me, the most honest part of the project: the **closing odds** are by far the most important feature. Removing them costs about **3.8 points**, while removing any other group (ELO, ranking, recent form, head-to-head, serve statistics) costs less than one point each. In other words, almost all of the signal my model uses is already contained in the bookmaker's probability, and the rest of the features add only a little on top.

Table 2: Ablation: test accuracy when one feature group is removed (full model 0.678).

Removed group	Accuracy	Δ (pt)
Bookmaker odds	0.641	−3.75
Head-to-head	0.670	−0.88
Recent form	0.672	−0.62
Ranking	0.673	−0.57
ELO	0.674	−0.45
Serve statistics	0.675	−0.33

Table 3: Confusion matrix on the test set.

	Predicted: P1 wins	Predicted: P2 wins
Actual: P1 wins	1421	673
Actual: P2 wins	682	1408

4.5 Confusion Matrix

Table 3 shows how the model classified the test matches. The matrix is almost symmetric, which is what you expect from the player-swap randomisation: the model has no built-in preference for "player 1" or "player 2" and the errors are split between the two classes.

4.6 Calibration

A high accuracy does not mean that the predicted *probabilities* are good. To check this, I grouped the test predictions in confidence bins and measured the actual win frequency inside each bin (Table 4).

The model is well calibrated above 0.60: when it says "80%" the player actually wins around 86% of the times, and when it says "90%" the player wins around 91%. The bin 0.50–0.60 is the hardest, as expected: predictions that close to 0.5 are basically uncertain matches, and on a small sample the observed win rate can swing.

4.7 Comparison with the BookMaker

The comparison that interested me the most is the one against the BookMaker. For every match in the test set with a closing odd, I have two predictions: the one of my model and the one implied by the BookMaker, both as probabilities of player 1 winning. I compared the two on the same matches, both on accuracy (which player is predicted as the winner) and on calibration of the probabilities.

The accuracy of my model and the accuracy of the BookMaker on the same matches are very close, with a difference inside the statistical noise. The calibration curves are also very similar. So, on the ATP main tour, the BookMaker is more or less as accurate as my model, and the probabilities implied by the closing odds already contain almost the same information that my features pull out from the public data.

Table 4: Calibration on the test set.

Predicted prob. bin	N matches	Observed win rate
0.50–0.60	79	0.49
0.60–0.70	89	0.70
0.70–0.80	20	0.70
0.80–0.90	49	0.86
0.90–1.00	11	0.91

4.8 What the model learnt

The ablation in Section 4.3 answers this directly. The single most useful input is the BookMaker-implied probability: removing it costs about 3.8 points, far more than any other group. The ELO ratings, ranking, recent form, head-to-head and serve statistics each contribute less than a point on their own, because they are partly redundant with each other and, above all, with the closing odds, which already summarise most of them. In tennis, most of the signal is captured by a small number of strong, well-known features, and the strongest single one is the market's own estimate.

5. Discussion

5.1 Strengths of the model

The model I built has three things going for it. First, it is built on a time-based split, so the test accuracy is a fair guess of how it would do on new, unseen matches. Second, the state-before-match rule makes sure that no feature contains information that was not yet available at the moment of the prediction. Third, the probabilities are calibrated: a "70%" from my model really means around 70% chance of winning, and this makes the output directly comparable to the BookMaker.

The other strong point is the clear gap with the amateur baseline. By going from around 63% to about 67–68%, I have shown that the limit of public attempts is not the data, it is the techniques used: more careful feature engineering, surface-specific ELO and calibration give a clear improvement on the same problem.

5.2 Why my model does not beat the BookMaker

The most interesting result is that, even with these techniques, my model does not really beat the closing odds. The reason, thinking about it after, is simple: the BookMaker has the same public data I have, plus the information that comes from how people are betting on the match. By the time the match starts, the closing odds have basically absorbed everything that ELO, rankings, surface, form and serve statistics can tell. A public model ends up using the same signal that the market already uses, so it can match the BookMaker but rarely go past it.

There is also a second reason, that has nothing to do with the model: tennis itself is noisy. Court conditions, momen-

tum, small injuries, mental state, all these things put a hard ceiling on what any pre-match predictor can do.

5.3 Limitations and future work

The first limitation is that I only used pre-match, free public data. I did not use in-play information (live score, live odds) and I did not use any non-public signal like injury reports. These would arrive later than the closing odds and would let the model react to news that the BookMaker has not used yet.

A second limitation is that the closing odds dataset only covers the ATP main tour. Smaller tours like the ATP Challenger circuit or the WTA might behave differently, but the public odds data needed for a fair comparison are much harder to find.

A natural direction for future work is to move to a setting where the state of the match changes over time, modelling tennis as a point-by-point Markov chain following Klaassen and Magnus [5]. In that setting, the probability that each player wins the next point on serve fully decides the probability of winning the match at any score, and the model can update the prediction during the match.

6. Conclusion

I built a tennis match prediction model on free public data, using ELO ratings, surface specific features, recent form, serve statistics and the BookMaker-implied probability as a feature. The final calibrated XGBoost model reaches around **67.6% accuracy** on a held-out test set of recent ATP matches, a clear improvement over the $\sim 63\%$ of the amateur baseline I started from.

When I compare the model directly with the BookMaker, however, the two are basically the same: the probabilities implied by the closing odds already contain the same predictive information that my model pulls out. For me this was the most interesting result of the project. It shows that the BookMaker is not just a price setter, but a serious predictive system, and that going past it would need something more: non-public data, or a different kind of model (in-play, point by point, or trained on a less liquid market).

Beyond the numbers, what I take from this project is a much more concrete idea of how a real prediction pipeline is built: how to avoid data leakage, how to split data in time, how to compare models in a fair way, and how to read calibration tables. The model itself is a small piece of code; the method around it is the part I will keep using in the next projects.

7. Code and data

The full code, the feature pipeline and the trained model are public on GitHub at `github.com/sussolinoss/sports-ml-predictions` (the trained weights are in the tagged release). The data is the public `tennis_atp` dataset [1] and the historical closing odds from tennis-data.co.uk [2], so every number in this paper can be reproduced from the same code and sources.

8. References

1. J. Sackmann, *Tennis match datasets*, GitHub repository `tennis_atp`.
2. *tennis-data.co.uk*, historical match data and closing odds.
3. A. Elo, *The Rating of Chessplayers, Past and Present*, 1978.
4. T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," *KDD*, 2016.
5. F. J. G. M. Klaassen and J. R. Magnus, "Are points in tennis independent and identically distributed?," *Journal of the American Statistical Association*, 2001.
6. G. W. Brier, "Verification of forecasts expressed in terms of probability," *Monthly Weather Review*, 1950.
7. Green Code (YouTube channel), tutorial on tennis match prediction with Random Forests.